

数据结构与算法面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

*已加入后端训练营的学员，可免费领一年面试鸭会员
领取方式见训练营文档

- ✓ 5000+中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序，随时随地都能刷
- ✓ 真题命中率高，持续更新

微信扫一扫 >>>

刷高频题拿高薪offer



数据结构

了解哪些数据结构？

- 数组：数组的内存空间是连续的，随机访问的时间复杂度是 $O(1)$ ，适用于需要按索引访问元素的场景，但是插入和删除元素较慢，时间复杂度是 $O(n)$
- 链表：链表是由节点组成，节点之间是分散存储的，内存不连续，每个节点存储数据和指向下一个节点的指针。适用于频繁插入和删除元素的场景，随机访问元素较慢。
- 栈：栈是一种后进先出的数据结构，只允许在栈顶进行插入和删除操作。
- 队列：队列是一种先进先出（FIFO）的数据结构，允许在队尾插入元素，在队首删除元素。
- 树：树是一种非线性数据结构，由节点和边组成，每个节点可以有多个子节点。树适用于表示层次关系的场景，例如文件系统、组织结构等。

数组和链表区别是什么？

- **访问效率**：数组可以通过索引直接访问任何位置的元素，访问效率高，时间复杂度为 $O(1)$ ，而链表需要从头节点开始遍历到目标位置，访问效率较低，时间复杂度为 $O(n)$ 。

- **插入和删除操作效率**：数组插入和删除操作可能需要移动其他元素，时间复杂度为 $O(n)$ ，而链表只需要修改指针指向，时间复杂度为 $O(1)$ 。
- ****缓存命中率**：**由于数组元素在内存中连续存储，可以提高CPU缓存的命中率，而链表节点不连续存储，可能导致CPU缓存的命中率较低，频繁的缓存失效会影响性能。
- **应用场景**：数组适合静态大小、频繁访问元素的场景，而链表适合动态大小、频繁插入、删除操作的场景

为什么数组查询的复杂度为 $O(1)$?

数组必须要内存中一块**连续**的空间，并且数组中必须存放**相同**的数据类型。

比如我们创建一个长度为 10，数据类型为整型的数组，在内存中的地址是从 1000 开始，那么它在内存中的存储格式如下。

9

74

1039

◦
◦
◦

2

241

1008

1

164

1004

0

35

1000

由于每个整型数据占据 4 个字节的内存空间，因此整个数组的内存空间地址是 1000 ~ 1039，根据这个，我们就可以轻易算出数组中每个数据的内存下标地址。

利用这个特性，我们只要知道了数组下标，也就是数据在数组中的位置，比如下标 2，就可以计算得到这个数据在内存中的位置 1008，从而对这个位置的数据 241 进行快速读写访问，时间复杂度为 $O(1)$ 。

说一下队列和栈的区别

主要区别在于元素的插入和删除方式以及元素的访问顺序。

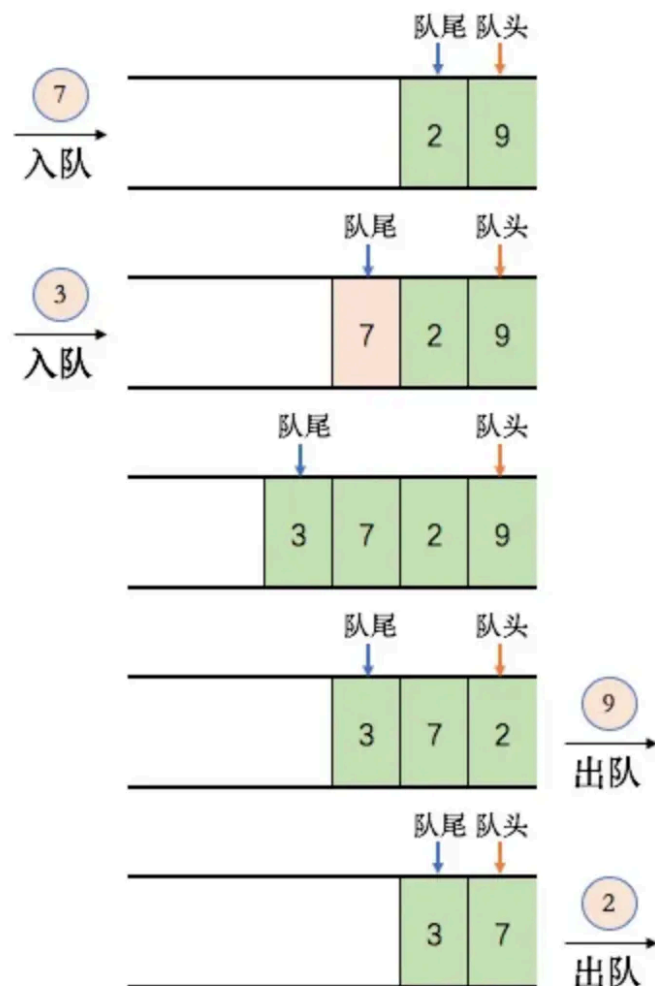
插入和删除方式：

- 队列：队列采用先进先出（FIFO）的方式，即新元素插入队尾，删除操作发生在队首。
- 栈：栈采用后进先出（LIFO）的方式，即新元素插入栈顶，删除操作也发生在栈顶。

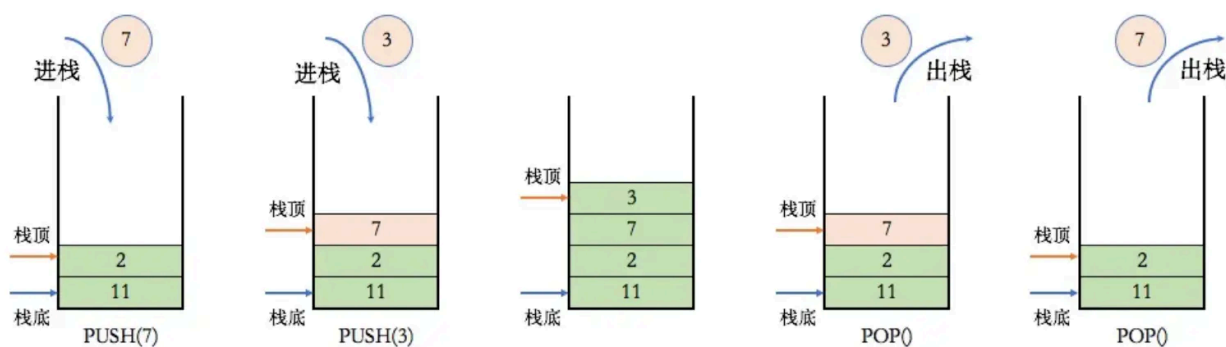
元素的访问顺序：

- 队列：队列的元素按照插入的顺序进行访问，先插入的元素先被访问到。
- 栈：栈的元素按照插入的顺序进行访问，但是最后插入的元素先被访问到。

队列适用于需要按照插入顺序进行处理的场景，例如任务调度；



而栈适用于需要维护最近操作状态的场景，例如函数调用。



如何使用两个栈实现队列？

使用两个栈实现队列的方法如下：

1. 准备两个栈，分别称为 `stackPush` 和 `stackPop`。
2. 当需要入队时，将元素压入 `stackPush` 栈。
3. 当需要出队时，先判断 `stackPop` 是否为空，如果不为空，则直接弹出栈顶元素；如果为空，则将 `stackPush` 中的所有元素依次弹出并压入 `stackPop` 中，然后再从 `stackPop` 中弹出栈顶元素。

作为出队元素。

4. 当需要查询队首元素时，同样需要先将 `stackPush` 中的元素转移到 `stackPop` 中，然后取出 `stackPop` 的栈顶元素但不弹出。
5. 通过上述方法，可以实现用两个栈来模拟队列的先进先出（FIFO）特性。

这种方法的时间复杂度为 $O(1)$ 的入队操作，均摊时间复杂度为 $O(1)$ 的出队和查询队首元素操作。

以下是使用两个栈实现队列的Java代码示例：

java

```
import java.util.Stack;

class MyQueue {
    private Stack<Integer> stackPush;
    private Stack<Integer> stackPop;

    public MyQueue() {
        stackPush = new Stack<>();
        stackPop = new Stack<>();
    }

    public void push(int x) {
        stackPush.push(x);
    }

    public int pop() {
        if (stackPop.isEmpty()) {
            while (!stackPush.isEmpty()) {
                stackPop.push(stackPush.pop());
            }
        }
        return stackPop.pop();
    }

    public int peek() {
        if (stackPop.isEmpty()) {
            while (!stackPush.isEmpty()) {
                stackPop.push(stackPush.pop());
            }
        }
        return stackPop.peek();
    }

    public boolean empty() {
        return stackPush.isEmpty() && stackPop.isEmpty();
    }
}
```

```

}

// 测试代码
public class Main {
    public static void main(String[] args) {
        MyQueue queue = new MyQueue();
        queue.push(1);
        queue.push(2);
        System.out.println(queue.peek()); // 输出 1
        System.out.println(queue.pop());  // 输出 1
        System.out.println(queue.empty()); // 输出 false
    }
}

```

平衡二叉树结构是怎么样子的？

使用二叉树搜索树的目的之一是缩短插入、删除、修改和查找（插入、删除、修改都包括查找操作）节点的时间。

关于查找效率，如果一棵树的高度为 h ，在最坏的情况，查找一个关键字需要对比 h 次，查找时间复杂度不超过 $O(h)$ 。一棵理想的二叉搜索树所有操作的时间可以缩短到 $O(\log n)$ （ n 是节点总数）。

然而 $O(h)$ 的时间复杂度仅为理想情况。在最坏情况下，搜索树有可能退化为链表。想象一棵每个结点只有右孩子的二叉搜索树，那么它的性质就和链表一样，所有操作（增删改查）的时间是 $O(n)$ 。

可以发现操作的复杂度与树的高度 h 有关。由此引出了平衡树，通过一定操作维持树的高度（平衡性）来降低操作的复杂度。

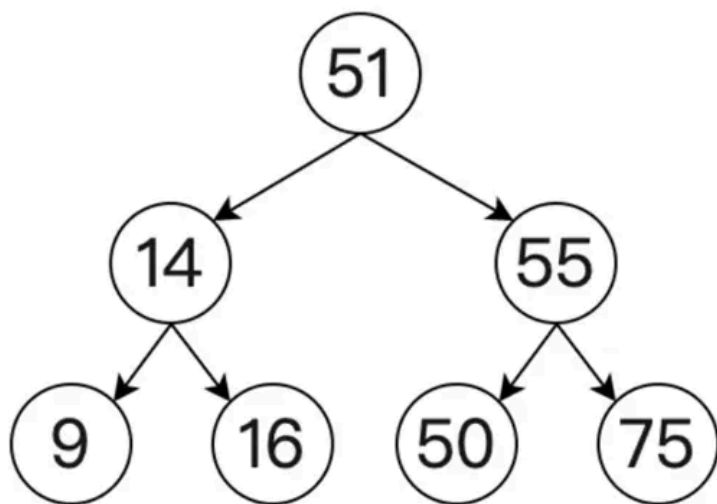
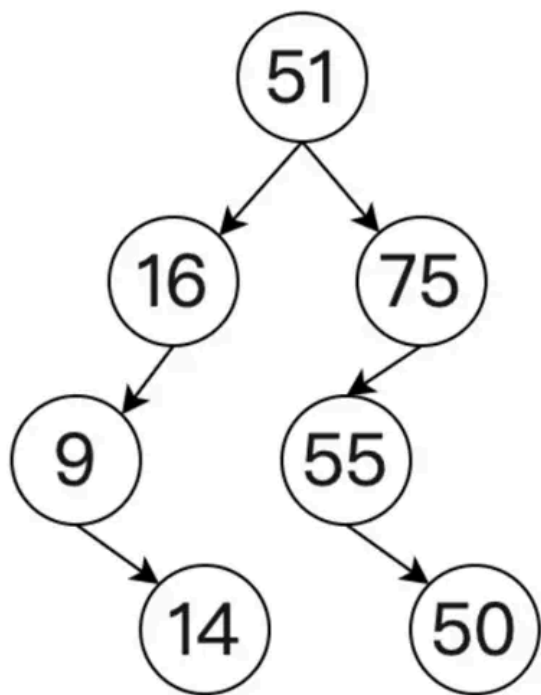
所谓的平衡树是指一种改进的二叉查找树，顾名思义平衡树就是将二叉查找树平衡均匀地分布，这样的好处就是可以减少二叉查找树的深度。

一般情况下二叉查找树的查询复杂度取决于目标节点到树根的距离（即深度），当节点的深度普遍较大时，查询的平均复杂度就会上升，因此为了实现更高效的查询就有了平衡树。

平衡二叉树平衡的特性：

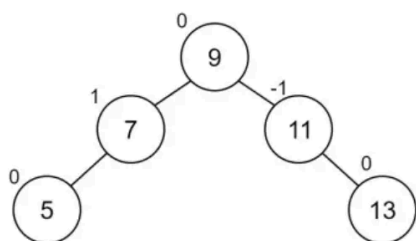
- 左右两个子树的高度差（平衡因子）的绝对值不超过1
- 左右两个子树都是一棵平衡二叉树

非平衡二叉树(左)和平衡二叉树(右)如下图所示：

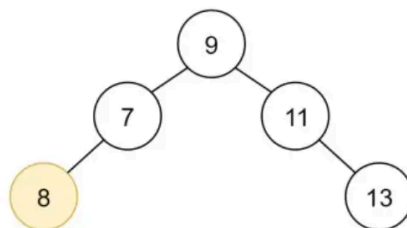


通过平衡的特性，可以有效的减少二叉树的深度，从而提高了查询的效率。

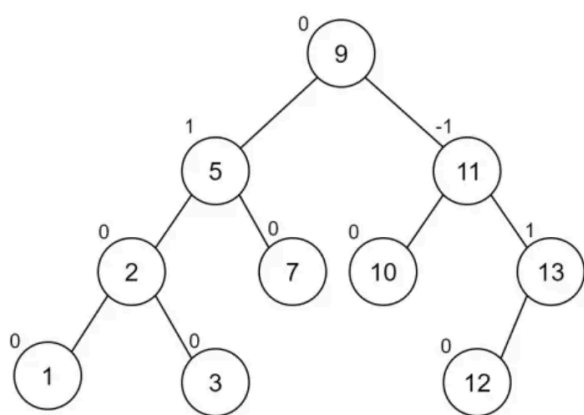
再来看看下图：



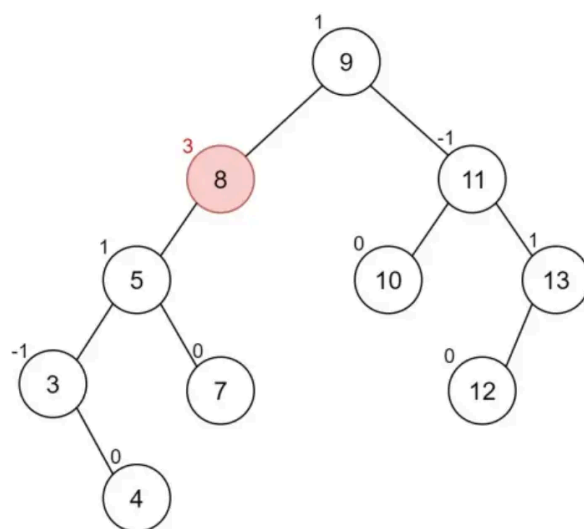
①平衡二叉树



②不是平衡二叉树



③平衡二叉树



④不是平衡二叉树

分析：

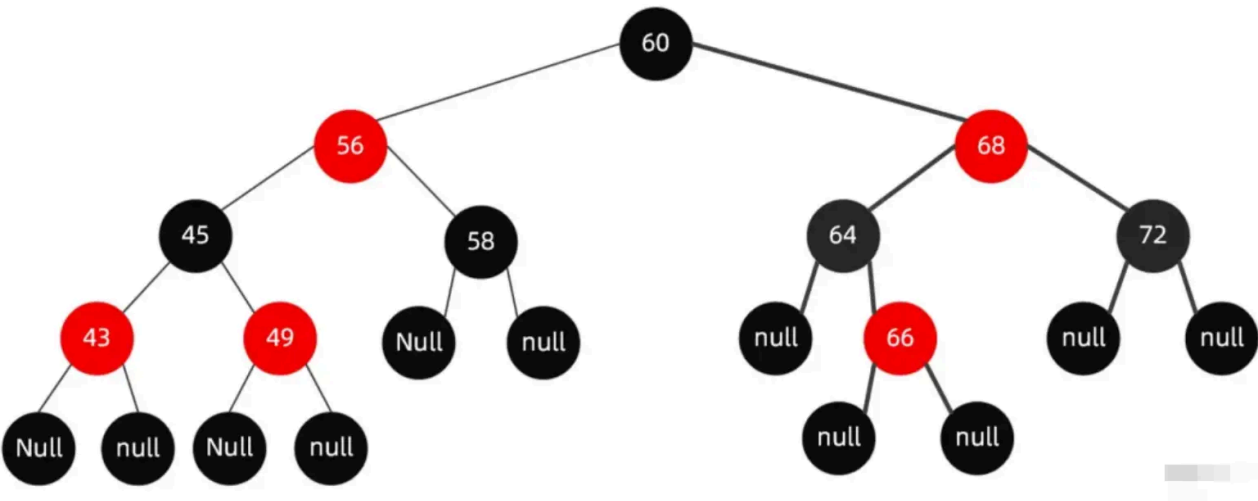
- 图一是一个平衡二叉树，它满足平衡二叉树的定义。
- 图二不是平衡二叉树，其原因并不是不满足平衡因子的条件，而是因为它不满足二叉搜索树的构成条件，这提醒我们平衡二叉树首先要是一棵二叉搜索树。
- 图三满足平衡二叉树的构成条件。
- 图 4 中的节点 (8) 平衡因子为 3，不满足平衡二叉树的要求。

红黑树说一下，跳表说一下？

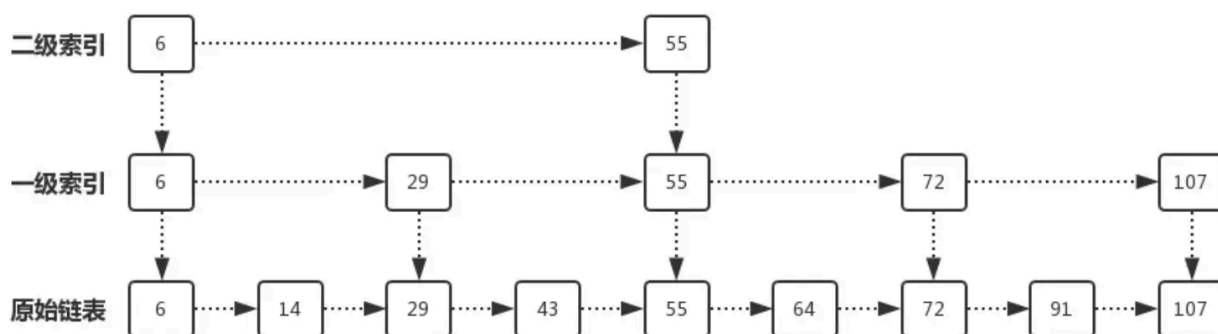
红黑树（Red-Black Tree）是一种自平衡的二叉搜索树，它在插入和删除操作后能够通过旋转和重新着色来保持树的平衡。红黑树的特点如下：

1. 每个节点都有一个颜色，红色或黑色。
2. 根节点是黑色的。
3. 每个叶子节点（NIL节点）都是黑色的。
4. 如果一个节点是红色的，则它的两个子节点都是黑色的。
5. 从根节点到叶子节点或空子节点的每条路径上，黑色节点的数量是相同的。

红黑树通过这些特性来保持树的平衡，确保最长路径不超过最短路径的两倍，从而保证了在最坏情况下的搜索、插入和删除操作的时间复杂度都为 $O(\log N)$ 。



跳表（Skip List）是一种基于链表的数据结构，它通过添加多层索引来加速搜索操作。



跳表的特点如下：

1. 跳表中的数据是有序的。
2. 跳表中的每个节点都包含一个指向下一层和右侧节点的指针。

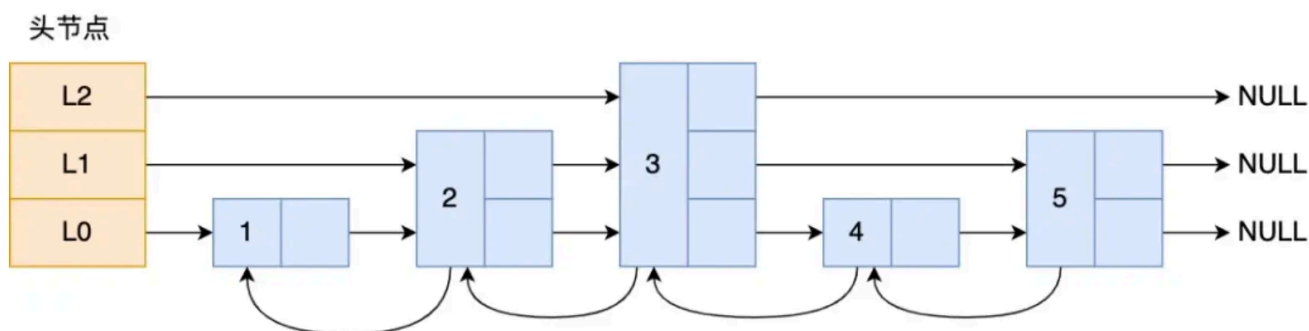
跳表通过多层索引的方式来加速搜索操作。最底层是一个普通的有序链表，而上面的每一层都是前一层的子集，每个节点在上一层都有一个指针指向它在下一层的对应节点。这样，在搜索时可以通过跳过一些节点，直接进入目标区域，从而减少搜索的时间复杂度。

跳表的平均搜索、插入和删除操作的时间复杂度都为 $O(\log N)$ ，与红黑树相比，跳表的实现更加简单，但空间复杂度稍高。跳表常用于需要高效搜索和插入操作的场景，如数据库、缓存等。

你知道什么地方用了红黑树和跳表吗？

- epoll 用了红黑树来保存监听的 socket
- redis 用了跳表来实现 zset

跳表时间复杂度？

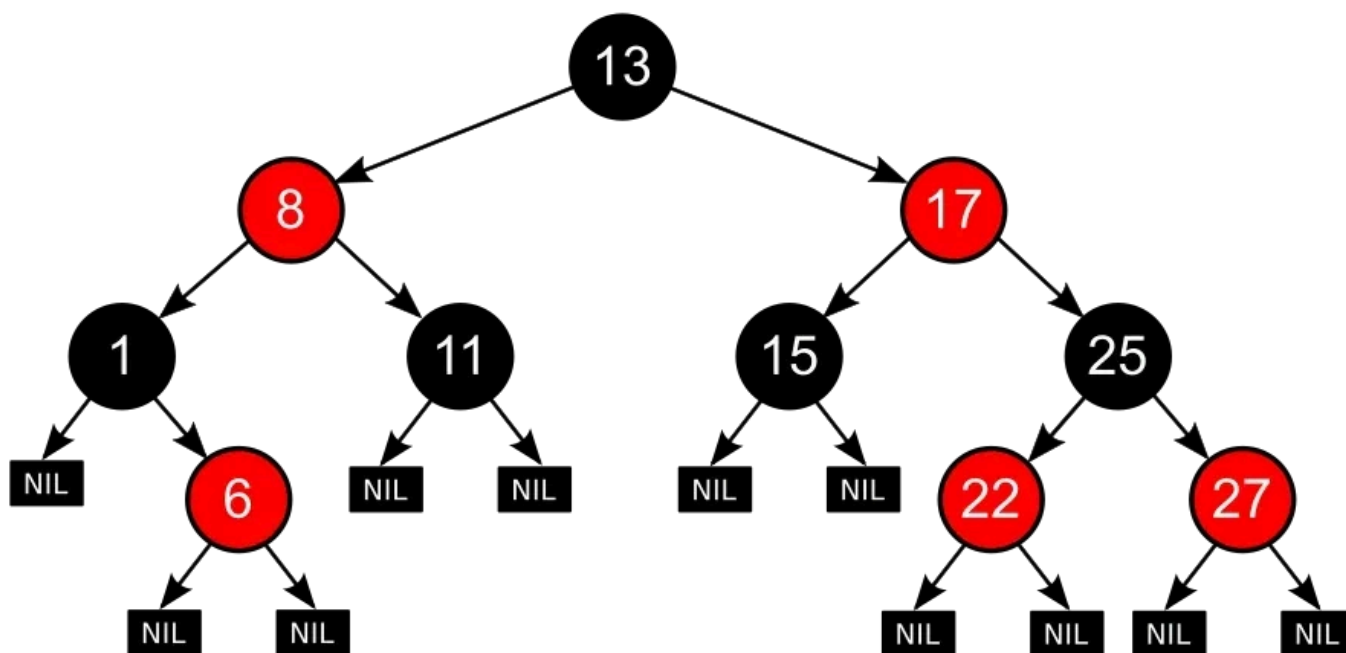


- 搜索操作的时间复杂度： $O(\log n)$ ，其中 n 是跳表中元素的数量。这是因为跳表中使用多级索引，可以通过跳跃的方式快速定位到目标元素所在的位置，从而将搜索的时间复杂度降低到对数级别。

- 插入和删除操作的时间复杂度： $O(\log n)$ ，其中 n 是跳表中元素的数量。与搜索操作类似，插入和删除操作也可以通过跳跃的方式快速定位到需要插入或删除的位置，并进行相应的操作。因此，插入和删除的时间复杂度也是对数级别的。

红黑树的数据结构介绍一下？

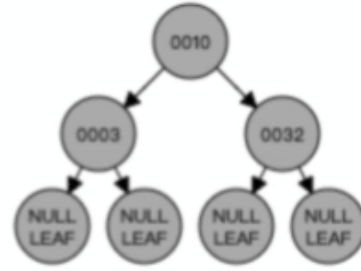
红黑树是一种自平衡的二叉查找树，



具有以下特点：

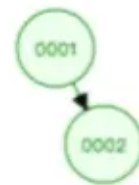
1. 每个节点要么是红色，要么是黑色。
2. 根节点是黑色。
3. 每个叶子节点（NIL节点）是黑色。
4. 如果一个节点是红色，则其子节点必须是黑色。
5. 从任一节点到其每个叶子节点的所有路径都包含相同数目的黑色节点。

红黑树的自平衡性质可以保证在进行插入、删除等操作后，树的高度保持在 $O(\log n)$ 内，从而保持了较高的查找、插入和删除效率。下面是红黑树插入节点的过程，这左旋右旋的操作，就是为了自平衡。



二叉树搜索最坏的时间复杂度，为什么会这样？以及用什么结果解决？

当每次插入的元素都是二叉查找树中最大的元素，二叉查找树就会退化成了链表，查找数据的时间复杂度变成了 $O(n)$ ，如下动图演示：

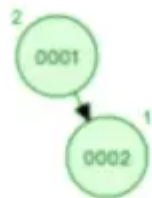


二叉查找树由于存在退化成链表的可能性，会使得查询操作的时间复杂度从 $O(\log n)$ 升为 $O(n)$ 。

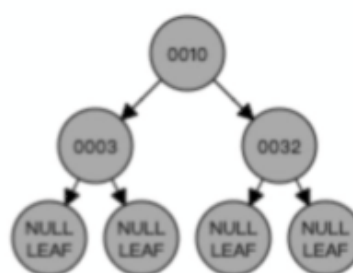
为了解决二叉查找树会在极端情况下退化成链表的问题，后面就有人提出**平衡二叉查找树（AVL 树）**。

主要是在二叉查找树的基础上增加了一些条件约束：**每个节点的左子树和右子树的高度差不能超过 1**。也就是说节点的左子树和右子树仍然为平衡二叉树，这样查询操作的时间复杂度就会一直维持在 $O(\log n)$ 。

下图是每次插入的元素都是平衡二叉查找树中最大的元素，可以看到，它会维持自平衡：



除了平衡二叉查找树，还有很多自平衡的二叉树，比如红黑树，它也是通过一些约束条件来达到自平衡，不过红黑树的约束条件比较复杂。下面是红黑树插入节点的过程，这左旋右旋的操作，就是为了自平衡。



B+树的特点是什么？

- B+树是一种自平衡的多路查找树，所有叶节点都位于同一层，保证了树的平衡，使得搜索、插入和删除操作的时间复杂度为对数级别的。
- 非叶节点仅包含索引信息，不存储具体的数据记录，它们只用来引导搜索到正确的叶节点。非叶节点的子树指针与关键字数量相同，每个子树指针指向一个子树，子树中的所有键值都在某个区间内。
- 所有数据记录都存储在叶节点中，且叶节点中的数据是按关键字排序的。叶节点包含实际的数据和关键字，它们是数据存储和检索的实体单元。叶节点之间通过指针相互链接，形成一个链表，便于范围查询和顺序遍历。

B+树和B树有什么不一样，B+树的叶子节点和非叶子节点有什么不一样，非叶子节点会不会存数据？

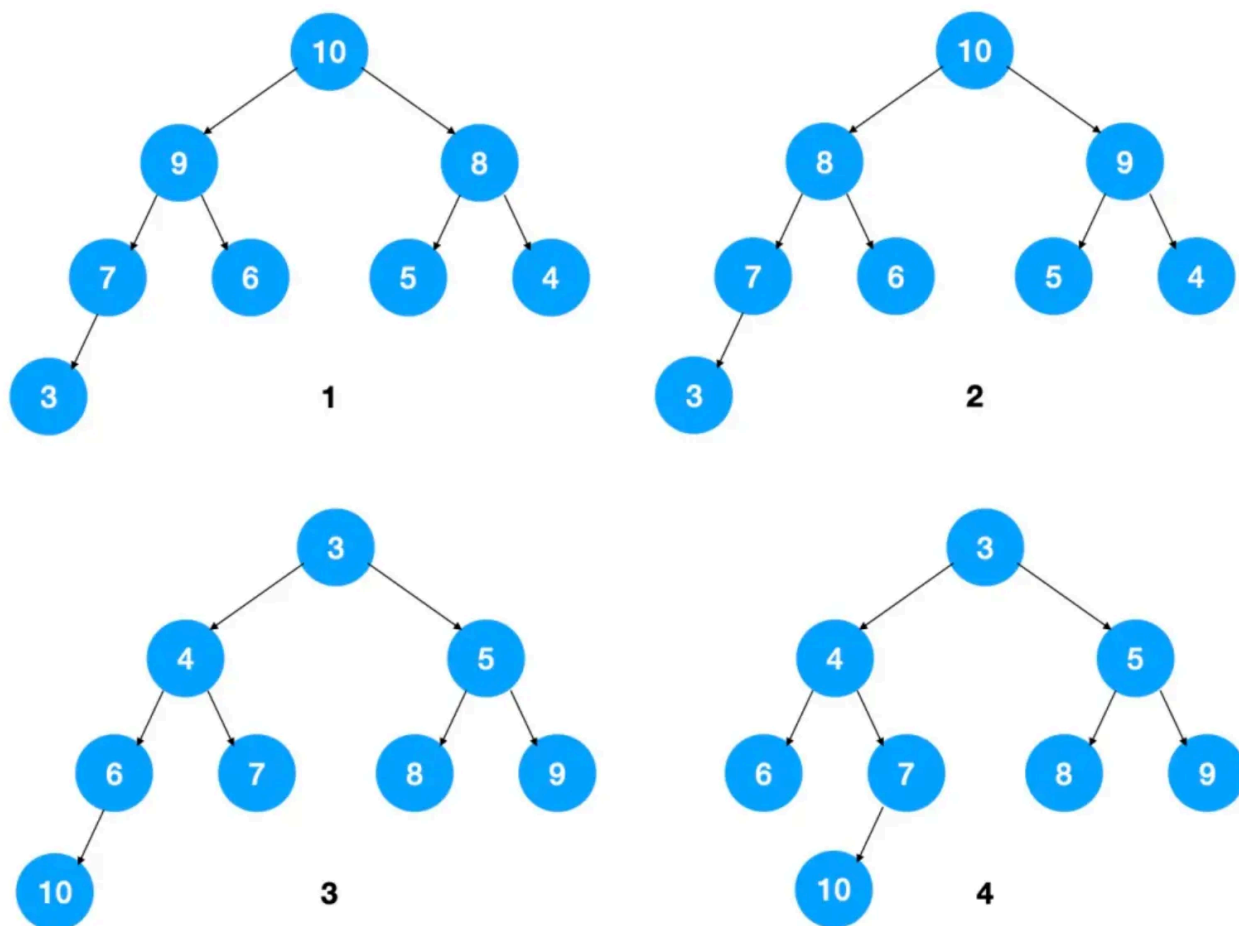
- **检索路径**：B树在查找数据时，可能在非叶子节点找到目标数据，路径长度不固定。即查找时可以在任意一个节点终止。B+树中所有数据都在叶子节点，查找数据时必须走到叶子节点，路径长度固定（均等）。即查找总是要到叶子节点结束。

- **叶子节点结构**: B树中叶子节点之间没有特别的链接, 彼此独立。B+树中叶子节点通过指针连接, 形成一个有序链表, 便于范围查询和顺序访问。
- **非叶子节点内容**: B树中非叶子节点存储数据和索引。B+树中非叶子节点只存储索引, 不存储实际数据。因此, 当数据量比较大时, 相对于B树, B+树的层高更少, 查找效率也就更高。
- ****高效地范围查询**: **B+树叶子节点采用的是双链表连接, 适合 MySQL 中常见的基于范围的顺序查找, 而 B 树在进行范围查询时需要进行中序遍历, 性能较差。

堆是什么?

堆是一颗完全二叉树, 这样实现的堆也被称为**二叉堆**。堆中节点的值都大于等于 (或小于等于) 其子节点的值, 堆中如果节点的值都大于等于其子节点的值, 我们把它称为**大顶堆**, 如果都小于等于其子节点的值, 我们将其称为**小顶堆**。

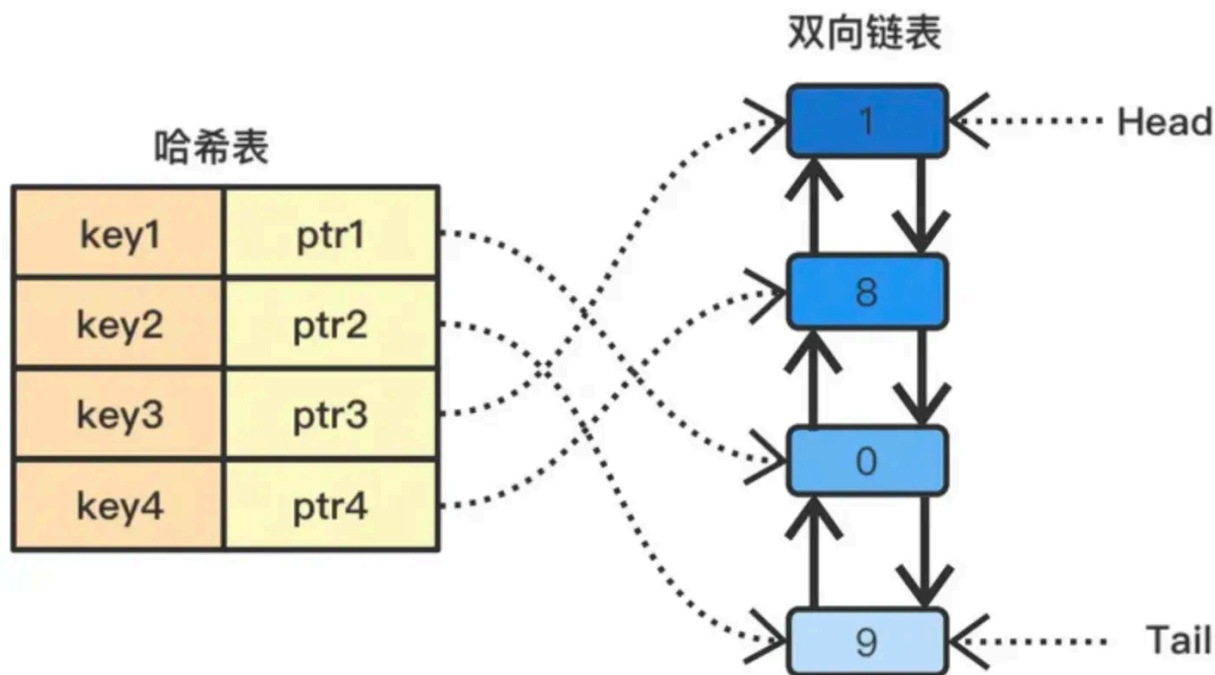
下图中, 1, 2 是大顶堆, 3 是小顶堆, 4 不是堆 (不是完全二叉树)



LRU是什么? 如何实现?

LRU 是一种缓存淘汰算法, 当缓存空间已满时, 优先淘汰最长时间未被访问的数据。

实现的方式是哈希表+双向链表结合。



具体实现步骤如下：

- 使用哈希表存储数据的键值对，键为缓存的键，值为对应的节点。
- 使用双向链表存储数据节点，链表头部为最近访问的节点，链表尾部为最久未访问的节点。
- 当数据被访问时，如果数据存在于缓存中，则将对应节点移动到链表头部；如果数据不存在于缓存中，则将数据添加到缓存中，同时创建一个新节点并插入到链表头部。
- 当缓存空间已满时，需要淘汰最久未访问的节点，即链表尾部的节点。

上面这种思想方式，LRU 算法可以在 $O(1)$ 的时间复杂度内实现数据的插入、查找和删除操作。每次访问数据时，都会将对应的节点移动到链表头部，保证链表头部的节点是最近访问的数据，而链表尾部的节点是最久未访问的数据。当缓存空间不足时，淘汰链表尾部的节点即可。

布隆过滤器怎么设计？时间复杂度？

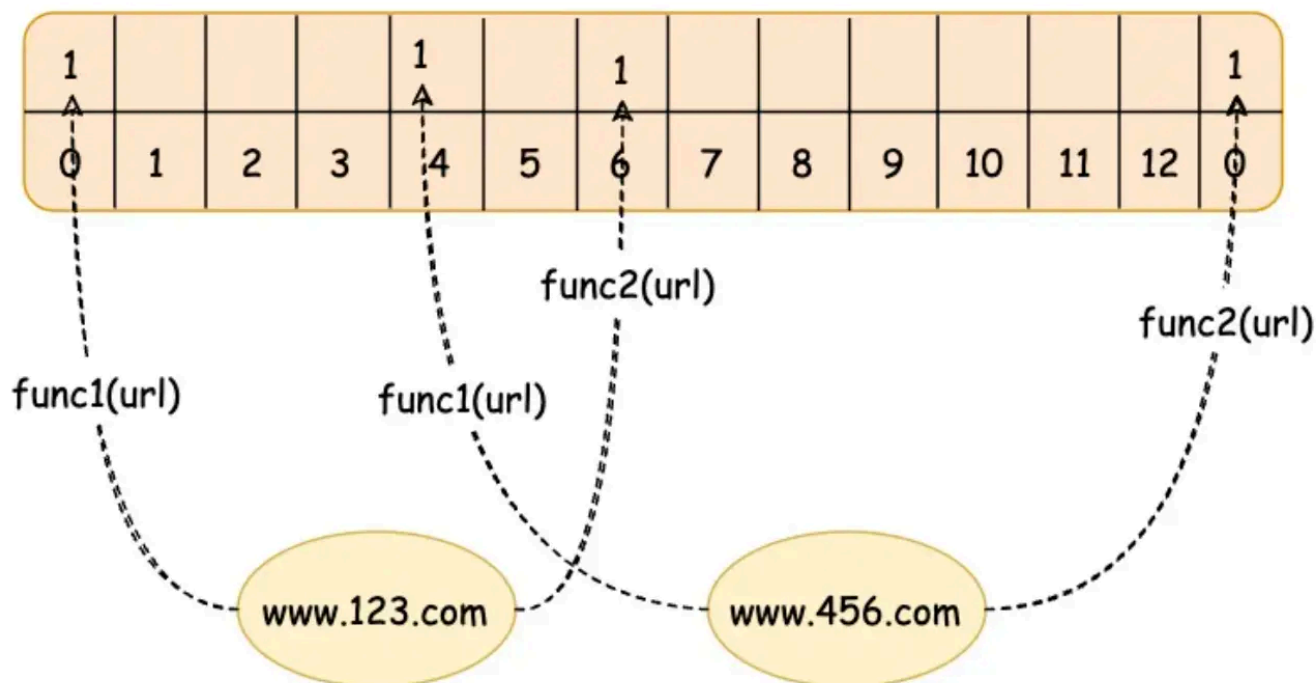
在开发过程中，经常要判断一个元素是否在一个集合中。假设你现在要给项目添加IP黑名单功能，此时你手上有大约 1 亿个恶意IP的数据集，有一个IP发起请求，你如何判断这个IP在你的黑名单中？

类似这种问题用Java自己的Collection和Map很难处理，因为它们存储元素本身，会造成内存不足，而我们只关心元素存不存在，对于元素的值我们并不关心，具体值是什么并不重要。

「布隆过滤器」可以用来解决类似的问题，具有运行快速，内存占用小的特点，它是一个保存了很长的二进制向量，同时结合 Hash 函数实现的。而高效插入和查询的代价就是，它是一个基于概率的数据结构，**只能告诉我们一个元素绝对不在集合内，对于存在集合内的元素有一定的误判率。**

布隆过滤器中总是会存在误判率，因为哈希碰撞是不可能百分百避免的。布隆过滤器对这种误判率称之为「**假阳性概率**」，即：**False Positive Probability**，简称为 fpp。在实践中使用布隆过滤器时可以自己定义一个 fpp，然后就可以根据布隆过滤器的理论计算出需要多少个哈希函数和多大的位数组空间。需要注意的是这个 fpp 不能定义为 100%，因为无法百分保证不发生哈希碰撞。

下图表示向布隆过滤器中添加元素 `www.123.com` 和 `www.456.com` 的过程，它使用了 `func1` 和 `func2` 两个简单的哈希函数。



其基本原理如下：

1. **初始化**：当我们创建一个布隆过滤器时，我们首先创建一个全由0组成的位数组（bit array）。同时，我们还需选择几个独立的哈希函数，每个函数都可以将集合中的元素映射到这个位数组的某个位置。
2. **添加元素**：在布隆过滤器中添加一个元素时，我们会将此元素通过所有的哈希函数进行映射，得到在位数组中的几个位置，然后将这些位置标记为1。
3. **查询元素**：如果我们要检查一个元素是否在集合中，我们同样使用这些哈希函数将元素映射到位数组中的几个位置，**如果所有的位置都被标记为1，那么我们就可以说该元素可能在集合中。如果有任何一个位置不为1，那么该元素肯定不在集合中。**

通过其原理可以知道，我们可以提高数组长度以及 hash 计算次数来降低误报率，但是相应的 CPU、内存的消耗也会相应地提高，会增加存储和计算的开销。因此，布隆过滤器的使用需要在误判率和性能之间进行权衡。布隆过滤器有以下两个特点：

- **只要返回数据不存在，则肯定不存在。**
- **返回数据存在，不一定存在。**

布隆过滤器的误判率主要来源于「**哈希碰撞**」。因为位数组的大小有限，不同的元素可能会被哈希到相同的位置，导致即使某个元素并未真正被加入过滤器，也可能因为其他已经存在的元素而让所有哈希函数映射的位都变为了1，从而误判为存在。这就是布隆过滤器的“假阳性”错误。在有限的数组长度中存放大量的数据，即便是再完美的 Hash 算法也会有冲突，所以有可能两个完全不同的 A、B 两个数据最后定位到的位置是一模一样的。这时拿 B 进行查询时那自然就是误报了。

****布隆过滤器的时间复杂度和空间复杂度：****对于一个 m（比特位个数）和 k（哈希函数个数）值确定的布隆过滤器，添加和判断操作的时间复杂度都是 $O(k)$ ，这意味着每次你想要插入一个元素或者查询一个元素是否在集合中，只需要使用 k 个哈希函数对该元素求值，然后将对应的比特位标记或者检查对应的比特位即可。

排序算法

说几个你懂的排序算法，并说明其时间空间复杂度

各种常用排序算法						
类别	排序方法	时间复杂度			空间复杂度	稳定性
		平均情况	最好情况	最坏情况	辅助存储	
插入排序	直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	shell排序	$O(n^{1.3})$	$O(n)$	$O(n^2)$	$O(1)$	不稳定
选择排序	直接选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(n\log_2n)$	$O(n\log_2n)$	$O(n\log_2n)$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(n\log_2n)$	$O(n\log_2n)$	$O(n^2)$	$O(n\log_2n)$	不稳定
归并排序		$O(n\log_2n)$	$O(n\log_2n)$	$O(n\log_2n)$	$O(1)$	稳定
基数排序		$O(d(r+n))$	$O(d(n+rd))$	$O(d(r+n))$	$O(rd+n)$	稳定
注：基数排序的复杂度中，r代表关键字的基数，d代表长度，n代表关键字的个数						

- 冒泡排序**：通过相邻元素的比较和交换，每次将最大（或最小）的元素逐步“冒泡”到最后（或最前）。时间复杂度：最好情况下 $O(n)$ ，最坏情况下 $O(n^2)$ ，平均情况下 $O(n^2)$ 。，空间复杂度： $O(1)$ 。
- 插入排序**：将待排序元素逐个插入到已排序序列的合适位置，形成有序序列。时间复杂度：最好情况下 $O(n)$ ，最坏情况下 $O(n^2)$ ，平均情况下 $O(n^2)$ ，空间复杂度： $O(1)$ 。
- 选择排序**（Selection Sort）：通过不断选择未排序部分的最小（或最大）元素，并将其放置在已排序部分的末尾（或开头）。时间复杂度：最好情况下 $O(n^2)$ ，最坏情况下 $O(n^2)$ ，平均情况下 $O(n^2)$ ，空间复杂度： $O(1)$ 。

- **快速排序** (Quick Sort) : 通过选择一个基准元素, 将数组划分为两个子数组, 使得左子数组的元素都小于 (或等于) 基准元素, 右子数组的元素都大于 (或等于) 基准元素, 然后对子数组进行递归排序。时间复杂度: 最好情况下 $O(n\log n)$, 最坏情况下 $O(n^2)$, 平均情况下 $O(n\log n)$, 空间复杂度: 最好情况下 $O(\log n)$, 最坏情况下 $O(n)$ 。
- **归并排序** (Merge Sort) : 将数组不断分割为更小的子数组, 然后将子数组进行合并, 合并过程中进行排序。时间复杂度: 最好情况下 $O(n\log n)$, 最坏情况下 $O(n\log n)$, 平均情况下 $O(n\log n)$ 。空间复杂度: $O(n)$ 。
- **堆排序** (Heap Sort) : 通过将待排序元素构建成一个最大堆 (或最小堆), 然后将堆顶元素与末尾元素交换, 再重新调整堆, 重复该过程直到排序完成。时间复杂度: 最好情况下 $O(n\log n)$, 最坏情况下 $O(n\log n)$, 平均情况下 $O(n\log n)$ 。空间复杂度: $O(1)$ 。

讲一下冒泡排序算法

冒泡排序: 通过相邻元素的比较和交换, 每次将最大 (或最小) 的元素逐步“冒泡”到最后 (或最前)

- 冒泡排序的最好时间复杂度出现在以下情况: 当待排序数组已经有序时, 即每个元素都比其前面的元素小, 那么在第一次遍历数组时就可以确定排序已经完成, 因此时间复杂度为 $O(n)$ 。
- 冒泡排序的时间复杂度为 $O(n^2)$ 。因为在排序过程中, 需要进行多次遍历和元素交换, 而每次遍历都需要比较相邻的元素并决定是否进行交换, 这种操作需要花费 $O(n)$ 的时间。因此, 冒泡排序的时间复杂度通常为 $O(n^2)$ 。

c

```
public class BubbleSort {
    // 冒泡排序算法
    public void bubbleSort(int[] arr) {
        int n = arr.length;

        // 外层循环控制比较轮数
        for (int i = 0; i < n - 1; i++) {
            // 内层循环进行两两比较并交换
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // 交换两个元素
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
```

```
int[] arr = {64, 34, 25, 12, 22, 11, 90};
BubbleSort bubbleSort = new BubbleSort();
bubbleSort.bubbleSort(arr);
System.out.println("Sorted array:");
for (int value : arr) {
    System.out.print(value + " ");
}
}
```

讲一下快排原理

快排使用了分治策略的思想，所谓分治，顾名思义，就是分而治之，将一个复杂的问题，分成两个或多个相似的子问题，在把子问题分成更小的子问题，直到更小的子问题可以简单求解，求解子问题，则原问题的解则为子问题解的合并。

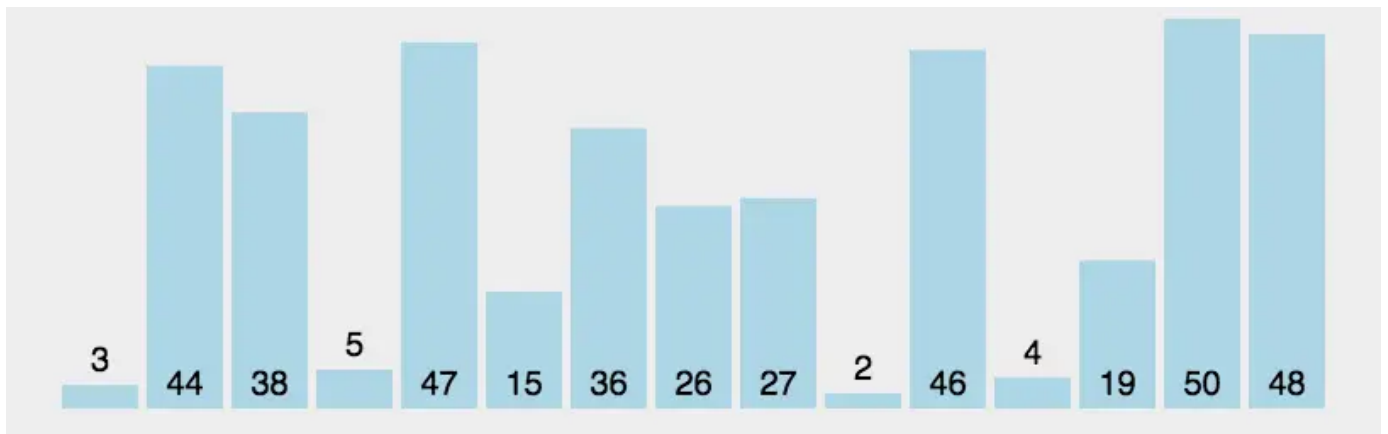
快排的过程简单的说只有三步：

- 首先从序列中选取一个数作为基准数
- 将比这个数大的数全部放到它的右边，把小于或者等于它的数全部放到它的左边（一次快排 partition）
- 然后分别对基准的左右两边重复以上的操作，直到数组完全排序

具体按以下步骤实现：

- 1，创建两个指针分别指向数组的最左端以及最右端
- 2，在数组中任意取出一个元素作为基准
- 3，左指针开始向右移动，遇到比基准大的停止
- 4，右指针开始向左移动，遇到比基准小的元素停止，交换左右指针所指向的元素
- 5，重复3，4，直到左指针超过右指针，此时，比基准小的值就都会放在基准的左边，比基准大的值会出现在基准的右边
- 6，然后分别对基准的左右两边重复以上的操作，直到数组完全排序

注意这里的基准该如何选择？最简单的一种做法是每次都是选择最左边的元素作为基准，但这对几乎已经有序的序列来说，并不是最好的选择，它将会导致算法的最坏表现。还有一种做法，就是选择中间的数或通过 `Math.random()` 来随机选取一个数作为基准。



代码实现：

```
public class QuickSort {  
    // 快速排序算法  
    public void quickSort(int[] arr, int low, int high) {  
        if (low < high) {  
            int pi = partition(arr, low, high);  
  
            // 递归排序左半部分  
            quickSort(arr, low, pi - 1);  
            // 递归排序右半部分  
            quickSort(arr, pi + 1, high);  
        }  
    }  
  
    // 划分函数，用于找到基准元素的正确位置  
    int partition(int[] arr, int low, int high) {  
        int pivot = arr[high]; // 选择最后一个元素作为基准  
        int i = low - 1; // 初始化较小元素的索引  
  
        for (int j = low; j < high; j++) {  
            if (arr[j] < pivot) {  
                i++;  
                // 交换元素  
                int temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
            }  
        }  
  
        // 将基准元素放到正确的位置  
        int temp = arr[i + 1];  
        arr[i + 1] = arr[high];  
        arr[high] = temp;  
    }  
}
```

java

```

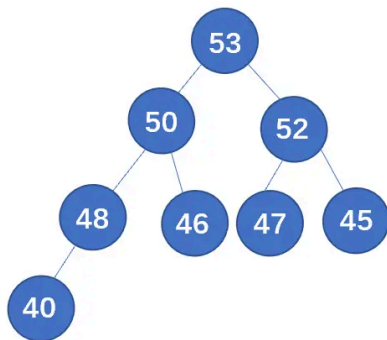
        return i + 1; // 返回基准元素的位置
    }

    public static void main(String[] args) {
        int[] arr = {10, 7, 8, 9, 1, 5};
        QuickSort quickSort = new QuickSort();
        quickSort.quickSort(arr, 0, arr.length - 1);
        System.out.println("Sorted array:");
        for (int value : arr) {
            System.out.print(value + " ");
        }
    }
}

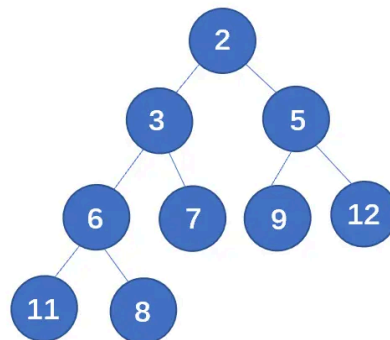
```

堆排序算法原理，稳定吗？

如果每个节点大于等于子树中的每个节点，我们称之为大顶堆，小于等于子树中的每个节点，我们则称之为小顶堆。



大顶堆



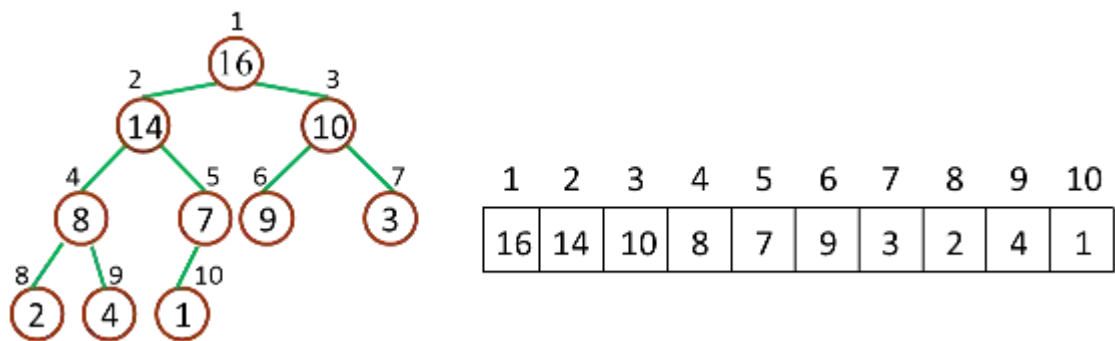
小顶堆

堆的要求：

- 必须是完全二叉树
- 堆中的每一个节点，都必须大于等于（或小于等于）其子树中每个节点的值。

堆通常是使用一维数组进行保存，节省空间，不需要存左右子节点的指针，通过下标就可定位左右节点和父节点。在起始位置为0的数组中：

- 父节点 i 的左子节点在 $(2i+1)$ 的位置
- 父节点 i 的右子节点在 $(2i+2)$ 的位置
- 子节点 i 的父节点在 $(i-1)/2$ 向下取整的位置



我们可以把堆排序的过程大致分为两大步骤，分别是建堆和排序。

- 建堆：建堆操作就是将一个无序的数组转化为最大堆的操作，首先将数组原地建一个堆。“原地”的含义就是不借助另一个数组，就在原数组上操作。我们的实现思路是从后往前处理数据，并且每个数据都是从上向下调整。
- 排序：建堆结束后，数组中的数据已经按照大顶堆的特性进行组织了，数组中的第一个元素就是堆顶，也就是最大的元素。我们把它和最后一个元素交换，那最大的元素就放到了下标为n的位置，时末尾元素就是最大值，将剩余元素重新堆化成一个大顶堆。继续重复这些步骤，直至数组有序排列

假设我们有一个数组 [4, 10, 3, 5, 1]，堆排序的过程如下：

1. 构建最大堆：[10, 5, 3, 4, 1]
2. 交换堆顶元素与最后一个元素：[1, 5, 3, 4, 10]
3. 调整剩余元素为堆：[5, 4, 3, 1]
4. 再次交换堆顶元素与最后一个元素：[1, 4, 3, 5]
5. 调整剩余元素为堆：[4, 3, 1]
6. 继续上述过程直到排序完成：[1, 3, 4, 5, 10]

算法实现：

```
public class HeapSort {
    // 堆排序方法
    public static void heapSort(int[] arr) {
        int n = arr.length;

        // 构建堆（重新排列数组）
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }

        // 依次从堆中提取元素
        for (int i = n - 1; i > 0; i--) {
            // 将当前根节点移动到末尾
```

```

        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // 在堆中调整
        heapify(arr, i, 0);
    }
}

// 通过索引i对数组arr的前n个元素进行堆调整
private static void heapify(int[] arr, int n, int i) {
    int largest = i; // 初始化最大值索引
    int left = 2 * i + 1; // 左孩子节点
    int right = 2 * i + 2; // 右孩子节点

    // 如果左孩子大于根节点
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    // 如果右孩子大于当前最大值
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // 如果最大值不是根节点
    if (largest != i) {
        int swap = arr[i];
        arr[i] = arr[largest];
        arr[largest] = swap;

        // 递归调整受影响的子树
        heapify(arr, n, largest);
    }
}

public static void main(String[] args) {
    int[] arr = {4, 10, 3, 5, 1};
    heapSort(arr);

    System.out.println("排序后的数组:");
    for (int i : arr) {
        System.out.print(i + " ");
    }
}

```



```
}  
}
```

现在我们来分析一下堆排序的时间复杂度、空间复杂度以及稳定性。

- 整个堆排序的过程中，只需要个别的临时存储空间，所以**堆排序是原地排序算法**。
- 堆排序包括建堆和排序两个操作，建堆的时间复杂度是 $O(n)$ ，排序过程时间复杂度是 $O(n\log N)$ 。所以，**堆排序的整个时间复杂度是 $O(n\log N)$** 。
- 因为在排序的过程中，存在将堆的最后一个节点跟堆顶互换的操作，所以有可能会改变值相同数据的原始相对顺序，所以**堆排序不是稳定的排序算法**。例如，假设我们有两个相同的元素A和B，且A在B前面。在构建和调整堆的过程中，B可能被移动到A的前面，从而破坏了它们原来的相对顺序。

归并排序和快速排序的使用场景

- 归并排序是稳定排序算法，适合排序稳定的场景；
- 快速排序是不稳定排序算法，不适合排序稳定的场景，快速排序是目前基于比较的内部排序中被认为是最好的方法，当待排序的关键字是随机分布时，快速排序的平均时间最短；

什么是排序稳定性？

排序算法的稳定性是指在排序过程中，当有多个具有相同关键字的元素时，这些元素在排序后的序列中保持它们原有的相对顺序。

换句话说，如果两个元素有相同的键值，那么在排序前，如果第一个元素在第二个元素之前，排序后第一个元素也应该在第二个元素之前。

具体来说，对于一个序列中的两个元素A和B，如果A和B的键值相同，且在排序前A在B之前，那么在排序后A仍然应该在B之前，算法才能被称为是稳定的。

例如，考虑一个包含姓名和年龄的列表：

```
[("Alice", 25), ("Bob", 25), ("Charlie", 20)]
```

如果排序算法是稳定的，那么在按年龄排序后，"Alice"和"Bob"的相对顺序不会改变：

```
[("Charlie", 20), ("Alice", 25), ("Bob", 25)]
```


但如果排序算法不稳定, "Alice"和"Bob"的相对顺序可能会在排序后改变:

```
[("Charlie", 20), ("Bob", 25), ("Alice", 25)]
```

在这种情况下, 排序算法就被认为是不稳定的。

稳定和不稳定排序算法有什么特点?

稳定排序算法的特点:

- 相同元素的相对位置不会改变, 排序后仍然保持原始顺序。
- 适用于需要保持元素间相对顺序关系的场景, 如按照年龄排序后按姓名排序。

不稳定排序算法的特点:

- 相同元素的相对位置可能会改变, 排序后不保证原始顺序。
- 可能会更快, 但不适用于需要保持元素间相对顺序关系的场景。

说说快排流程, 时间复杂度

快速排序的流程如下:

- 从数组中选择一个基准元素 (通常是数组中间位置的元素)。
- 将数组分成两部分, 小于基准元素的放在左边, 大于基准元素的放在右边。
- 递归地对左右两部分进行快速排序。

快速排序的时间复杂度为 $O(n \log n)$, 其中 n 为数组的长度。最坏情况下时间复杂度为 $O(n^2)$, 发生在每次选择的基准元素都是最大或最小值时。平均情况下时间复杂度为 $O(n \log n)$, 效率较高。

快排为什么时间复杂度最差是 $O(n^2)$

主要是因为每次划分时选择的基准元素不合适导致的。当每次选择的基准元素都是当前子数组中的最大或最小元素时, 就会导致每次划分只能减少一个元素, 而不是均匀地分成两部分, 从而造成时间复杂度达到 $O(n^2)$ 。

这种情况通常发生在数组已经有序或基本有序的情况下。为了避免最坏情况发生, 可以通过随机选择基准元素或者使用三数取中法等策略来提高快速排序的性能。

快排这么强, 那冒泡排序还有必要吗?

冒泡排序在一些特定场景下仍然有其优势，比如：

- 对于小规模数据或基本有序的数据，冒泡排序可能比快速排序更简单、更直观。
- 冒泡排序是稳定排序算法，相对于快速排序的不稳定性，在某些情况下可能更适合要求稳定性的场景。
- 冒泡排序是原地排序算法，不需要额外的空间，适合空间复杂度要求严格的场景。

如果要对一个很大的数据集，进行排序，而没办法一次性在内存排序，这时候怎么办？

可以使用外部排序来解决，基本思路分为两个阶段。

部分排序阶段。

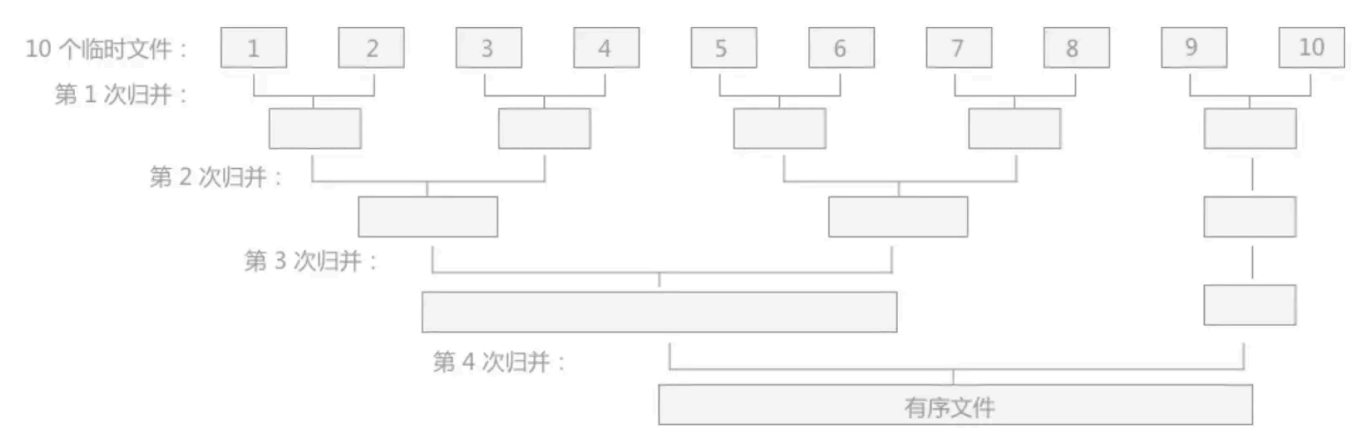
我们根据内存大小，将待排序的文件拆成多个部分，使得每个部分都是足以存入内存中的。然后选择合适的内排序算法，将多个文件部分排序，并输出到容量可以更大的外存临时文件中，每个临时文件都是有序排列的，我们将其称之为一个“顺段”。

在第一个阶段部分排序中，由于内存可以装下每个顺段的所有元素，可以使用快速排序，时间复杂度是 $O(n\log n)$ 。

归并阶段

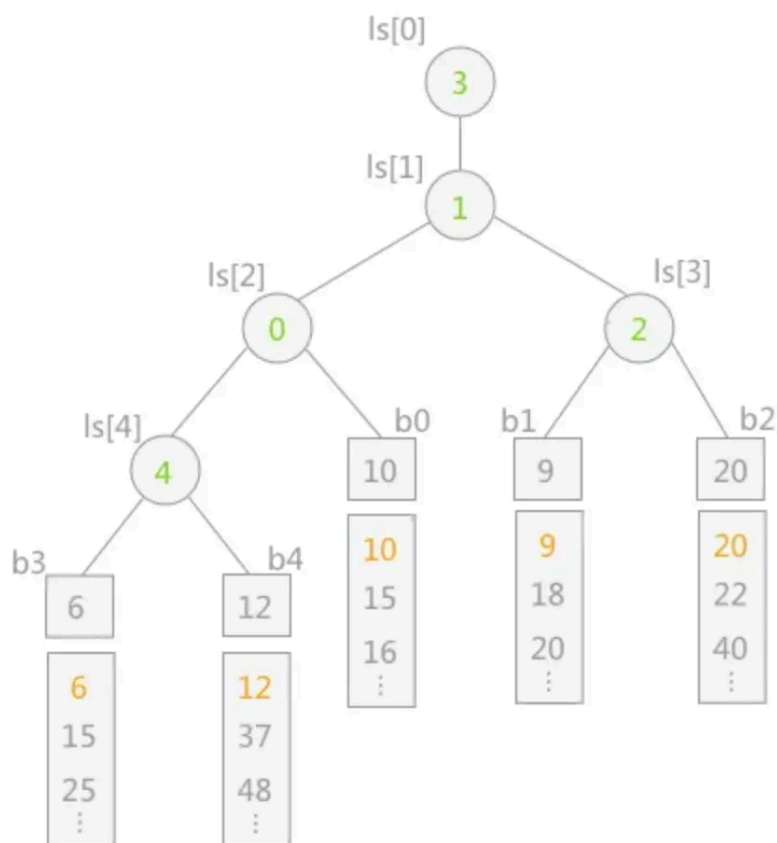
我们对前面的多个“顺段”进行合并，思想和归并排序其实是一样的。以 2 路归并为例，每次都两个连续的顺段合并成一个更大的顺段。

因为内存限制，每次可能只能读入两个顺段的部分内容，所以我们需要一部分一部分读入，在内存里将可以确定顺序的部分排列，并输出到外存里的文件中，不断重复这个过程，直至两个顺段被完整遍历。这样经过多层的归并之后，最终会得到一个完整的顺序文件。



归并阶段有个非常大的时间消耗就是 IO，也就是输入输出。最好就是让归并的层数越低越好，为了降低降低归并层数，可以使用**败者树**。

败者树中的非终端结点中存储的是胜利（左右孩子相比较，谁最小即为胜者）的一方；而败者树中的非终端结点存储的是失败的一方。而在比较过程中，都是拿胜者去比较。



现在有了败者树的加持，多路归并排序就可以比较高效地解决外部排序的问题了。

大致思路就是：

- 先用内排序算法（比如快速排序），尽可能多的加载源文件，将其变成 n 个有序顺段。
- 在内存有限的前提下每 k 个文件为一组，每次流式地从各个文件中读取一个单词，借助败者树选出字典序最低的一个，输出到文件中，这样就可以将 k 个顺段合并到一个顺段中了；反复执行这样的操作，直至所有顺段被归并到同一个顺段。



面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

*已加入后端训练营的学员，可免费领一年面试鸭会员
领取方式见训练营文档

- ✓ 5000+中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序，随时随地都能刷
- ✓ 真题命中率高，持续更新

微信扫一扫 >>>

刷高频题拿高薪offer



最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群



登录后才能发表评论 | 支持 Markdown 语法

使用 GitHub 帐号登录后发表评论

[使用 GitHub 登录](#)

[登录后查看评论](#)